Staffline Portal

Backend API specification

126 endpoints in 13 modules: 114 needed to replace the app's demo data layer, 12 more for server-side calculation, real data volumes and file generation. Every request and response model with its fields, every permission and business rule the server must enforce, background jobs, and a build order.

Version 1.3 · 26 October 2026 · for the backend team
Derived from the Flutter app: `lib/features/*/domain/` (repository interfaces and models) and `lib/core/permissions/`.

Contents

1. How the app talks to the backend
2. Endpoint count by module
3. Conventions (URLs, auth, JSON, errors, paging, concurrency)
4. Authorization: roles, permissions and data scope
5. Endpoints by module (with fields, permission and rules)
6. Data models
7. Background jobs and server-side events
8. Build order and integration checklist
9. Decisions the backend team must make

1. How the app talks to the backend

The Flutter app never calls HTTP directly from screens. Each feature has a **repository interface** in `lib/features/<feature>/domain/`. Today each one is implemented by an in-memory `Demo…Repository`. To connect the backend, the mobile/web developer writes an HTTP implementation of the same interface; no screen changes. This document therefore lists, for every endpoint, which repository method it **replaces**.

Interface (app)	Methods	Module in this document
`AuthRepository`	14	Authentication and session
`OrganizationRepository`	11	Organization and branches
`EmployeeRepository`	13	Employees, custom fields and import
`AttendanceRepository`	12	Attendance
`LeaveRepository`	10	Leave
`CalendarRepository`	6	Holidays and schedules
`AccessRepository`	4	Roles and access
`SettingsRepository`	2	Settings
`NotificationRepository`	7	Notifications
`AuditRepository`	2	Audit log
`ReportRepository`	2	Reports, exports and dashboards
`PlatformRepository`	1	Platform statistics (super admin)

Important — the server is the security boundary. The app hides pages and buttons a role may not use, but that is only for usability. Every endpoint must check the caller's permission, data scope and the business rules in this document, and must never trust IDs, roles, times or totals sent by the client.

Where the app calculates today. Leave balances and routing (`LeaveBook`), attendance day status (`AttendanceBook`), dashboards, reports and import validation run inside the app on fully loaded data. Endpoints marked **N** move that work to the server; they are needed once an organization has more than a few hundred people, and they are the only way to make the rules tamper-proof.

2. Endpoint count by module

Module	Required (R)	Server-side (N)	Total
Authentication and session	18	0	18
Organization and branches	12	0	12
Employees, custom fields and import	20	4	24
Attendance	13	3	16
Leave	15	2	17
Holidays and schedules	7	0	7
Roles and access	9	0	9
Settings	5	0	5
Notifications	4	0	4
Audit log	4	0	4
Reports, exports and dashboards	3	3	6
Platform statistics (super admin)	3	0	3
Files	1	0	1
Total	**114**	**12**	**126**

By HTTP method: GET 51, POST 47, PUT 23, PATCH 1, DELETE 4.

R = needed to replace the demo repositories one-for-one. **N** = recommended so the server calculates (balances, day status, reports, dashboards, import checks) and files are generated server-side.

3. Conventions

Topic	Rule
Base URL	`https://api.<domain>/api/v1`. Everything inside an organization is under `/orgs/{orgId}`; the server checks the caller is an active member of that organization.
Format	JSON, UTF-8, field names in camelCase exactly as in the data models. Enums are sent as the lower-camel names listed (e.g. `noticePeriod`).
Dates and times	Instants: ISO 8601 UTC (`2026-10-14T08:40:00Z`). Calendar dates: `YYYY-MM-DD`. Times of day: minutes after midnight (int). Time zones: IANA ids. The server converts to the branch time zone when grouping work days.
Authentication	Access token (JWT, 15 min) in `Authorization: Bearer`. Refresh token in an httpOnly, Secure, SameSite=Strict cookie (web) or secure storage (mobile), rotated on each refresh. "Keep me signed in" extends the refresh lifetime.
Re-authentication	Saving roles and security settings requires `X-Reauth: <token>` from `POST /auth/reauthenticate` (valid 5 minutes). Logged in the audit log.
Paging	List endpoints take `page` (1-based) and `pageSize` (default 50, max 200) or `cursor`; response `{items, total, page, pageSize}` or `{items, nextCursor}`.
Concurrency	Editable records carry `version`. Send it back on PUT (or `If-Match`). A stale version → **409 conflict**; nothing is overwritten.
Idempotency	`Idempotency-Key` header on punches, imports, exports and invitations. A repeat with the same key returns the first result.
Reasons	Mutations that the app asks a reason for (status changes, adjustments, role changes, reveals, corrections) carry `reason`; it is stored in the audit entry.
Masking	Restricted fields (mobile, personal email, date of birth, registration number, IP) are returned masked (e.g. `98••••••21`) unless the caller has the restricted right and organization scope.
Rate limits	Sign-in, two-step, forgot-password and resend endpoints: per IP and per account. 429 with `Retry-After`.

Error format

```
HTTP 422
{
  "error": {
    "code": "validation",          // see table
    "message": "Check the highlighted fields.",   // safe to show
    "fieldErrors": { "workEmail": "This email belongs to Komal Desai (EMP-0231).",
                     "custom.bloodGroup": "Choose a value." },
    "until": null,                 // set for "locked"
    "reference": "req_8a234"       // support id, always set
  }
}
```

HTTP	code	App Failure class	When
400 / 422	validation	ValidationFailure (fieldErrors)	Input rejected; field keys match model field names
401	unauthenticated	(sign-out)	Missing or expired token
403	forbidden	PermissionFailure	Permission, scope or self-approval rule
404	notFound	NotFoundFailure	Missing **or out of the caller's scope** (never reveal it exists)
409	conflict	ConflictFailure	Stale version, duplicate, rule conflict (e.g. branch still has employees)
410	expired	ExpiredFailure	Reset link, invitation or code expired or used
423	locked	LockedFailure (until)	Too many attempts
429	rateLimited	NetworkFailure	Rate limit
5xx	unexpected	UnexpectedFailure (reference)	Server error; include reference

4. Authorization: roles, permissions and data scope

A permission is a key `module.action`. Modules: `organization, employees, employeeFields, attendance, leave, holidays, settings, roles, reports, importExport, auditLog`. Actions: `view, create, edit, deactivate, approve, correct, export`. Each role has a set of grants and one data scope. Custom roles store their own grants and a base system role.

Module	Owner	HR Admin	Branch Mgr	Attendance Mgr	Leave Approver	Employee
organization	all	view	view	—	—	—
employees	all	all	view create edit export	—	—	—
employeeFields	all	view edit	—	—	—	—
attendance	all	all	view approve correct export	view approve correct export	—	—
leave	all	all	view create approve export	—	view approve	—
holidays	all	all	view	—	—	—
settings	all	view edit	—	—	—	—
roles	all	—	—	—	—	—
reports	all	view export	view export	—	—	—
importExport	all	all	—	—	—	—
auditLog	all	view export	—	—	—	—

Data scope	Roles	Server filter on every read, write and export
organization	Owner, HR Admin	All records of the organization
assignedBranches	Branch Manager, Attendance Manager	`record.branchId` ∈ `member.branchIds` (branches list too)
ownTeam	Leave Approver	Employees whose `managerId` = the member's employee id
ownRecords	Employee	Only the member's own employee record

Rules every endpoint applies

- **Self-service** (punch, own attendance, own leave, own profile, own notifications) is allowed for every member with an employee record, whatever the role.
- **Owner-only:** organization edit, owner profile, branch create/edit/status/setup, roles (all), security and data settings, revealing masked values. Owner-only grants can never be given to another role; the last owner cannot lose the role.
- **No self-approval:** nobody decides their own leave request or attendance correction.
- **Out of scope = 404**, not 403, so the server never confirms a record exists.
- **Every mutation** writes an audit entry (section 7) in the same transaction.
- Role or permission changes apply on the member's next request: keep grants server-side (or short-lived tokens) and invalidate on change.

5. Endpoints by module

Paths are relative to `/api/v1`; ~ stands for `/orgs/{orgId}`. **Replaces** = the app's repository method. **R/N**: see section 2. Field names refer to section 6.

5.1 Authentication and session (18)

Public sign-in flows, plus the signed-in member's session. Replaces `AuthRepository`.

#	Method	Path	Permission	Input	Output	Replaces	
1	POST	`/auth/sign-in`	Public	identifier (work email or employee ID), password, keepSignedIn	account, requiresTwoStep, challengeToken (if two-step) or tokens	`signIn`	R
2	POST	`/auth/two-step/verify`	challengeToken	code, isBackupCode, trustDevice	tokens, backupCodesLeft (when a backup code was used)	`verifyTwoStep`	R
3	POST	`/auth/refresh`	Refresh cookie	—	new access token (rotates refresh token)	–	R
4	POST	`/auth/sign-out`	Signed in	—	204; refresh token revoked	–	R
5	POST	`/auth/reauthenticate`	Signed in	password	reauthToken (5 min), sent as `X-Reauth` on sensitive saves	`"Confirm it's you"`	R
6	POST	`/auth/register`	Public	fullName, email, password, phone?	202; 6-digit code emailed	`register`	R
7	POST	`/auth/verify-email`	Public	email, code	account + tokens	`verifyEmail`	R
8	POST	`/auth/verify-email/resend`	Public	email	202	`resendVerificationCode`	R
9	POST	`/auth/password/forgot`	Public	identifier	Always 202 (never reveals if the account exists)	`requestPasswordReset`	R
10	GET	`/auth/password/reset/{token}`	Public	—	maskedEmail	`openResetLink`	R
11	POST	`/auth/password/reset`	Public	token, newPassword	204	`resetPassword`	R
12	GET	`/auth/invitations/{token}`	Public	—	Invitation	`openInvitation`	R
13	POST	`/auth/invitations/{token}/accept`	Public	password	account + tokens	`acceptInvitation`	R
14	GET	`/me`	Signed in	—	UserAccount	`accountById`	R
15	GET	`/me/memberships`	Signed in	—	OrganizationMembership[]	`membershipsOf`	R
16	PUT	`/me/last-organization`	Signed in	organizationId	204	`rememberLastOrganization`	R
17	GET	`~/session`	Member of org	—	AppSession (role, grants, scope, branchIds, employeeId)	`session building`	R
18	POST	`~/access-requests`	Member of org	path (page that was refused)	ownerName	`requestAccess`	R

Rules the server must enforce

- Wrong sign-in details return one generic message that never reveals whether the account exists.
- Two-step is required by the organization's policy (Security settings: optional / admins / everyone). Default: Owner, HR Admin, Branch Manager, Attendance Manager.

- Three wrong codes or `lockAfterAttempts` wrong passwords lock sign-in for `lockMinutes` (default 15). Respond 423 with `until`.
- Reset links: one use, valid `resetMinutes` (default 30). Invitations valid `inviteDays` (default 7). Expired or used → 410.
- Password rules: at least 10 characters (or the organization minimum, 8–64), a number, upper and lower case letters, and not containing the person's name or email. Hash with Argon2id or bcrypt.
- "Trust this device" lasts 30 days. Backup codes are single-use.
- Idle timeout (`adminIdleMinutes`, `employeeIdleDays`) and session length are enforced by token lifetimes.
- A deactivated membership cannot be opened (403).

5.2 Organization and branches (12)

Replaces `OrganizationRepository`.

#	Method	Path	Permission	Input	Output	Replaces	
1	POST	`/orgs`	Signed in, verified email	name, employeeRange, logo? (upload id)	Organization; caller becomes Owner	`createOrganization`	R
2	GET	`~`	organization.view (all members get name + logo)	—	Organization	`organization`	R
3	PUT	`~`	organization.edit	Organization fields + version	Organization	`updateOrganization`	R
4	GET	`~/owner-profile`	organization.edit	—	OwnerProfile or 204	`ownerProfile`	R
5	PUT	`~/owner-profile`	organization.edit	OwnerProfile	OwnerProfile	`saveOwnerProfile`	R
6	GET	`~/branches`	organization.view (scoped)	status?	Branch[] (only branches in scope)	`branches`	R
7	GET	`~/branches/{branchId}`	organization.view (scoped)	—	Branch; 404 if out of scope	–	R
8	POST	`~/branches`	organization.edit	Branch (basics)	Branch	`createBranch`	R
9	PUT	`~/branches/{branchId}`	organization.edit	Branch (any section: hours, attendance, holidays, leave, roles, reminders) + version	Branch	`updateBranch`	R
10	POST	`~/branches/{branchId}/status`	organization.edit	status (active | inactive)	Branch; 409 while employees are assigned	`setBranchStatus`	R
11	GET	`~/setup-bookmark`	organization.edit	—	SetupBookmark or 204	`setupBookmark`	R
12	PUT	`~/setup-bookmark`	organization.edit	branchId, step	204	`saveSetupBookmark`	R

Rules the server must enforce

- Branch name and code are unique within the organization (409 / field error). Branch and organization phones: 10 digits including the area code, not starting 0/1. PIN: 6 digits, not starting 0.
- PIN code is 6 digits; each open day closes after it opens; at least one open day.
- `employeeCount` is computed from employee records (inactive included, terminated excluded), never stored.
- Branch time zone is an IANA id; null means "inherit organization time zone" (default Asia/Kolkata).
- Setup progress is computed from saved data (weights basics 25, attendance 20, holidays 15, leave 20, roles 15, reminders 5) — the server may return it as `setupPercent`.

5.3 Employees, custom fields and import (24)

Replaces `EmployeeRepository`.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	~/employees	employees.view (scoped)	search, branchId, department, status, type, sort (name | department | joined | code), ascending, page, pageSize	Page<Employee> (masked restricted fields)	employees	R
2	GET	~/employees/{id}	employees.view (scoped)	—	Employee incl. activity; 404 if out of scope	employees	R
3	POST	~/employees	employees.create	Employee fields, sendInvite	Employee	create	R
4	PUT	~/employees/{id}	employees.edit	Employee fields + version, notes[]	Employee	update	R
5	POST	~/employees/{id}/status	employees.deactivate	status, reason, lastWorkingDay?	Employee	setStatus	R
6	POST	~/employees/branch-move	employees.edit	employeeIds[], branchId, managerId?, effectiveFrom	movedCount	changeBranch	R
7	POST	~/employees/invitations/resend	employees.create	employeeIds[]	sentCount (signed-in people skipped)	resendInvites	R
8	POST	~/employees/{id}/password-reset	employees.edit	—	202; reset link emailed	sendPasswordReset	R
9	GET	~/me/profile	Self	—	Own Employee record	selfEmployee	R
10	PATCH	~/me/profile	Self	mobile, emergencyContact, preferredLanguage (per member)	Employee	My profile edit	R
11	PUT	~/employees/{id}/photo	employees.edit or self	upload id	Employee	photo upload	R
12	GET	~/departments	Signed in	—	string[] (today a fixed list in the app)	–	N
13	GET	~/employee-field-settings	Signed in (forms need it)	—	EmployeeFieldSettings	fieldSettings	R
14	PUT	~/employee-field-settings	employeeFields.edit	rules {field: required | optional | hidden} + version	EmployeeFieldSettings	saveFieldSettings	R
15	GET	~/custom-fields	employees.view	includeArchived?	CustomField[]	customFields	R
16	POST	~/custom-fields	employeeFields.edit	CustomField	CustomField	saveCustomField	R
17	PUT	~/custom-fields/{id}	employeeFields.edit	CustomField, renamedOptions {old: new}	CustomField	saveCustomField	R
18	PUT	~/custom-fields/order	employeeFields.edit	fieldIds[]	204	reorderCustomFields	R
19	POST	~/employee-imports/validate	importExport.create + employees.create	file (upload id) or rows, mapping, dateOrder, duplicatePolicy	ImportPlan (per-row result and messages)	client-side today	N
20	POST	~/employee-imports	importExport.create + employees.create (+ employees.edit to update)	fileName, plan rows, skipped[], sendInvites	ImportRecord	commitImport	R
21	GET	~/employee-imports	importExport.view	—	ImportRecord[]	imports	R
22	GET	~/employee-imports/{id}	importExport.view	—	ImportRecord	imports	R

#	Method	Path	Permission	Input	Output	Replaces	
23	GET	~/employee-imports/{id}/error-report	importExport.view	—	.xlsx file	client-side today	N
24	GET	~/employee-imports/template	importExport.create	—	.xlsx template	client-side today	N

Rules the server must enforce

- One validator for create, edit and import. Field errors use the keys below in `fieldErrors`.
- `code`: required, upper-cased, `^[A-Z0-9][A-Z0-9-]{1,19}$`, unique. `workEmail`: valid, unique; required only when an invitation is sent.
- **Field settings (D-73):** built-in fields follow the organization's rules. Locked as required (422 if a rule tries to change them): code, firstName, lastName, branchId, joiningDate, type, shift, managerId. workEmail may be required or optional, never hidden; probation and weeklyOff optional or hidden. A required field that is empty → `fieldErrors[key] = "Enter …"`; hidden fields are not required and keep saved values. personalEmail, dateOfBirth and emergencyContact are only enforced for callers who can see restricted fields. Imports apply the same rules; required fields with no import column become row warnings.
- `mobile`: Indian mobile — 10 digits starting 6–9 after removing +91 / 91 / 0, spaces, dashes and brackets; store normalized as +91XXXXXXXXXX. firstName/lastName: letters (any script), space . ' -, max 50. personalEmail must differ from workEmail. emergencyContact: must contain a name and a valid phone, not the employee's own mobile, max 80. dateOfBirth: not future, age ≥ 18 on joiningDate, ≤ 80. joiningDate ≤ today + 6 months. designation 2–60 chars with a letter (D-78).
- `personalEmail`: valid if given. `designation` and `department` (from the list) required by default; `branchId` (active branch) always required. `shift` must be one of the branch's shifts.
- `managerId` is required (except HR Admin and Branch Manager, who may report to the owner), must be employed, and must work at the same branch.
- Custom fields: `custom.<key>` errors; required fields enforced; per-audience view/edit access (hrAdmin, branchManager, attendanceManager, self) applied when reading and writing.
- Someone who still manages employed people cannot be deactivated or terminated (409). Terminated people are excluded from default lists and head counts but keep all history.
- Branch managers can only create people in, and move people within, their own branches.
- Import: duplicates matched by employee ID. "Update existing" changes only mapped, non-empty columns and never branch, manager or joining date. Ambiguous dates must be rejected unless `dateOrder` is given. Each row is saved on its own and re-checked; a failed row is listed, never half-saved.
- Every change appends to the employee's activity (`created | updated | moved | status | account`) and to the audit log.

5.4 Attendance (16)

Replaces `AttendanceRepository`. Today the app loads raw data and calculates day records itself (`AttendanceBook`); with real volumes the server should return calculated days.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	~/attendance	attendance.view (scoped) or self	from, to, branchId?, employeeId?	AttendanceData: punches, corrections, requests, reminders	load	R
2	POST	~/attendance/punches	Self (own employee record)	multipart: kind (punchIn | punchOut | breakStart | breakEnd), source (app | web | kiosk), selfie (JPEG, required when the branch has selfieCheck and the person isn't exempt); header Idempotency-Key = requestId	Punch (server time) incl. face {result, score}	punch	R

#	Method	Path	Permission	Input	Output	Replaces	
3	POST	~/attendance/corrections	attendance.correct (scoped)	employeeId, workDate, changes[] {kind, at, replaces?}, reason, requestId?	Correction (approved, or pending when a second approval is needed)	correct	R
4	POST	~/attendance/corrections/{id}/decision	attendance.approve (not own)	approve, note?	Correction	decide	R
5	POST	~/attendance/correction-requests	Self	workDate, kind, proposedAt, message	CorrectionRequest	request	R
6	POST	~/attendance/correction-requests/{id}/reject	attendance.correct (scoped)	reason	CorrectionRequest	rejectRequest	R
7	POST	~/attendance/reminders	attendance.correct (scoped)	items[] {employeeId, workDate}	sentCount	remind	R
8	GET	~/attendance/face-reviews	attendance.approve (scoped)	status (waiting | decided), branchId?, cursor	punches to review with selfie and enrolled photo URLs	punchesToReview	R
9	POST	~/attendance/punches/{punchId}/face-review	attendance.approve (scoped, not own)	accept, note (required to reject, ≥ 3 chars)	FaceReview	reviewFacePunch	R
10	PUT	~/me/face	Self	multipart: consent = true, photos[3] (JPEG)	FaceProfile (pending)	enrolFace	R
11	DELETE	~/me/face	Self	—	204; face data and templates deleted	deleteFace	R
12	POST	~/employees/{id}/face/decision	Owner or HR Admin (organization scope + employees.edit), not own	approve, reason (required to reject)	FaceProfile	decideFace	R
13	PUT	~/employees/{id}/face-exemption	Owner or HR Admin	reason (≥ 3 chars), or null to end it	FaceExemption or 204	setFaceExemption	R
14	GET	~/attendance/days	attendance.view (scoped) or self	date or from/to, branchId?, department?, employeeId?, exceptionsOnly?	DayRecord[] + AttendanceCounts	AttendanceBook (client today)	N
15	GET	~/attendance/monthly	attendance.view (scoped)	month, branchId?, department?	per-employee month grid + totals	Monthly summary (client today)	N
16	GET	~/attendance/missing-punches	attendance.view (scoped)	branchId?, overdueOnly?	open sessions with age	Missing punches (client today)	N

Rules the server must enforce

- Original punches are never edited or deleted. A correction adds events (or replaces one by `replaces` id) and counts only when approved.
- A correction made by the employee's own manager needs a second approver (HR). Nobody approves their own correction (403).
- Employees may request a correction only within 3 days of the work day, one open request per day.
- Punch time is the server clock in UTC, never the device clock. A second punch-in while a session is open → 409. Retrying with the same Idempotency-Key returns the original punch.
- Work days are grouped in the branch's time zone. Night shift: punches before noon belong to the previous day. A punch-out more than 20 h after punch-in starts a new session with a missing punch-in.
- Day status order: missing punch → half day (late beyond `halfDayLateMinutes`, default 120, or worked ≤ half of expected) → left early (`earlyLeaveMinutes`, 30) → short shift (>1 h short) → late (after `graceMinutes`, 10) → present. Expected = shift minus a 60-minute break.
- Approved full-day leave turns an unpunched scheduled day into "on leave"; punches on a leave day win.

- **Selfie punches (D-70):** when the branch has `attendance.selfieCheck` and the person has no exemption, a punch without a selfie → 422. The server runs face detection, a passive liveness check and 1:1 matching against the approved enrolment. Results: `verified` (above the match threshold), `notMatched`, `noFace` (no face or liveness failed), `notEnrolled`. The punch is **always saved**; anything but verified waits for review.
- A face review is final (409 if repeated). A rejected punch is excluded from every calculation and export; the original punch and the review stay in the audit trail.
- **Enrolment (D-71):** requires `consent = true` and three photos; replaces any earlier enrolment and sets it back to pending. Only Owner/HR approve; nobody approves their own. Deleting removes photos and face templates immediately.
- Store face templates and photos encrypted, separate from other employee data; never return them in employee lists or exports. Only reviewers in scope and the employee can view a selfie, and each view is audited. Punch selfies are deleted after 90 days.

5.5 Leave (17)

Replaces `LeaveRepository`. The server must apply the same routing, counting and self-approval rules as `LeaveBook` in the app.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	`~/leave/types`	Signed in	includeInactive?	LeaveType[]	`load`	R
2	POST	`~/leave/types`	leave.edit	LeaveType	LeaveType	`saveType`	R
3	PUT	`~/leave/types/{key}`	leave.edit	LeaveType	LeaveType	`saveType`	R
4	GET	`~/leave/policy`	leave.view	—	LeavePolicy	`load`	R
5	PUT	`~/leave/policy`	leave.edit	LeavePolicy + version	LeavePolicy	`savePolicy`	R
6	GET	`~/leave/requests`	leave.view (scoped) or self	status?, typeKey?, branchId?, employeeId?, from?, to?, search?, page	Page<LeaveRequest>	`load`	R
7	GET	`~/leave/requests/{id}`	leave.view (scoped) or self	—	LeaveRequest	`load`	R
8	POST	`~/leave/requests/preview`	Self	typeKey, from, to, half	days, balanceAfter, approvers, teamOff, blockingErrors[]	`LeaveBook (client today)`	N
9	POST	`~/leave/requests`	Self	typeKey, from, to, half, reason	LeaveRequest with routed steps	`submit`	R
10	PUT	`~/leave/requests/{id}`	Self (pending or changes requested)	typeKey, from, to, half, reason	LeaveRequest (back to pending; amendedFrom kept)	`amend`	R
11	POST	`~/leave/requests/{id}/decision`	leave.approve, assigned step, not own	approve, note (required to reject, ≥ 3 chars)	LeaveRequest	`decide`	R
12	POST	`~/leave/requests/{id}/request-changes`	leave.approve, assigned step	note	LeaveRequest	`requestChanges`	R
13	POST	`~/leave/requests/{id}/withdraw`	Self (pending)	—	LeaveRequest	`withdraw`	R
14	POST	`~/leave/requests/{id}/cancel`	Self (approved, not started, policy allows)	—	LeaveRequest	`cancel`	R
15	GET	`~/leave/adjustments`	leave.view (scoped)	employeeId?, year?	LeaveAdjustment[]	`load`	R
16	POST	`~/leave/adjustments`	leave.edit	employeeId, typeKey, days (±), effective, reason, carry	LeaveAdjustment	`adjust`	R
17	GET	`~/leave/balances`	leave.view (scoped) or self	year, employeeId?, branchId?, search?	per employee: LeaveBalance[]	`LeaveBook (client today)`	N

Rules the server must enforce

- Balance = granted (upfront and prorated, or accrued monthly after `startsAfterMonths`) + carried + adjustments − approved − pending. Never stored as one number.
- Days are counted from the work calendar: weekly offs and holidays are skipped unless the type counts them. Adding a holiday later gives approved days back.
- Routing: step 1 = reporting manager (HR when the branch approves centrally, or no manager). Optional step 2 from the most specific route: type › branch › default (`SecondStep`: none, hrIfDays, branchManagerIfDays, hrAlways).
- Nobody is routed to, or may decide, their own request — such a step goes to HR. HR never decides the same request twice.
- Who decides: Owner/HR any pending step except their own; Branch Manager any request in their branches; Leave Approver only steps assigned to them.
- Validation: type eligibility (employment type, branch, department), min/max days, half days allowed, notice days, backdating (≤ `backdateDays`, max 30), longest request, blackout dates, negative balance only if `allowNegative` and within `negativeLimit`.
- Optional holidays use a type with `optionalHolidaysOnly`, limited to `optionalHolidaysPerYear` (default 1).

5.6 Holidays and schedules (7)

Replaces `CalendarRepository`. Branch holiday lists from setup stay on the branch.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	`~/holidays`	Signed in (filtered to own branch for employees)	year, branchId?	CompanyHoliday[]	`holidays`	R
2	POST	`~/holidays`	holidays.create	CompanyHoliday	CompanyHoliday + affectedLeaveRequests	`saveHoliday`	R
3	PUT	`~/holidays/{id}`	holidays.create	CompanyHoliday	CompanyHoliday	`saveHoliday`	R
4	DELETE	`~/holidays/{id}`	holidays.create	—	204	`deleteHoliday`	R
5	POST	`~/holidays/import`	holidays.create	rows[] CompanyHoliday	CompanyHoliday[]	`importHolidays`	R
6	GET	`~/schedule-assignments`	holidays.view	level?, branchId?	ScheduleAssignment[]	`assignments`	R
7	POST	`~/schedule-assignments`	holidays.create	ScheduleAssignment	ScheduleAssignment	`addAssignment`	R

Rules the server must enforce

- Past holidays cannot be added, edited or deleted (409). Range up to 14 days. Overlapping holidays for the same branch are refused.
- Scope: `branchIds` empty = all branches; `optional` = optional holiday.
- Schedules: most specific assignment in effect wins (employee › group › branch › organization); a person's own weekly off beats group/branch. `effectiveFrom` cannot be in the past.

5.7 Roles and access (9)

Replaces `AccessRepository`.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	`~/roles`	roles.view	—	RoleDefinition[] with holder counts	`load`	R
2	GET	`~/roles/{id}`	roles.view	—	RoleDefinition	`load`	R
3	POST	`~/roles`	roles.create + X-Reauth	RoleDefinition, reason	RoleDefinition	`saveRole`	R

#	Method	Path	Permission	Input	Output	Replaces	
4	PUT	~/roles/{id}	roles.edit + X-Reauth	RoleDefinition + version, reason	RoleDefinition	saveRole	R
5	GET	~/role-assignments	roles.view	roleId?, search?	members with role, scope, branchIds, teamSize	load	R
6	PUT	~/role-assignments/{memberKey}	roles.edit	roleId, branchIds[], reason	RoleAssignment	assign	R
7	GET	~/invitations	employees.create (scoped)	state (waiting | expired | bounced | accepted)	invitation rows	Invitations page	R
8	DELETE	~/invitations/{memberKey}	employees.create	—	204 (revoke)	revoke	R
9	DELETE	~/invitations/{memberKey}/bounce	employees.create	—	204	clearBounce	R

Rules the server must enforce

- System roles (owner, hrAdmin, leaveApprover, employee) are read-only; they can be duplicated.
- Owner-only permissions (organization create/deactivate, roles create/edit/deactivate) cannot be granted to any other role (422).
- The last owner cannot lose the Owner role (409).
- A branch-scoped role needs at least one branch.
- Changing a role updates the employee record (for leave routing) and takes effect on the member's next request: re-issue or invalidate their access token.

5.8 Settings (5)

Replaces `SettingsRepository`.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	~/settings	settings.view	—	OrgSettings (security, data, branding)	load	R
2	PUT	~/settings/security	Owner + X-Reauth	SecuritySettings	SecuritySettings	save	R
3	PUT	~/settings/data	Owner	DataSettings	DataSettings	save	R
4	PUT	~/settings/branding	settings.edit	BrandingSettings	BrandingSettings	save	R
5	PUT	~/settings/attendance	settings.edit	per-branch AttendanceRules	Branch[]	branch rules	R

Rules the server must enforce

- `minPasswordLength` 8–64; `inviteDays`, `resetMinutes`, lock-out and idle values must be positive.
- Security and Data & export are owner-only (D-58).

5.9 Notifications (4)

Replaces `NotificationRepository`. send, `queueEmail` and outbox become server-internal.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	~/me/notifications	Signed in	category?, unreadOnly?, cursor	AppNotification[] + unreadCount	inbox	R
2	POST	~/me/notifications/read	Signed in	id? (none = mark all read)	204	markRead	R
3	GET	~/notification-prefs	settings.view	—	NotificationPrefs	prefs	R
4	PUT	~/notification-prefs	settings.edit	NotificationPrefs	NotificationPrefs	savePrefs	R

Rules the server must enforce

- Notifications are created by the server from the same mutations (leave submitted/decided/withdrawn, missing punch, correction decided, repeated lateness, invitation, profile changed, invite bounced, holiday soon/added).
- Delivery follows the preference matrix (event × employee/manager/HR × in-app/email). Nobody is notified about their own action. No passwords, attachments or restricted fields in any message.

5.10 Audit log (4)

Replaces `AuditRepository`. `record` becomes server-internal: every mutating endpoint writes its own entry.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	`~/audit-events`	auditLog.view	days, actor?, areas[]?, entityType?, branchId?, search?, cursor	AuditEvent[] (masked)	`events`	R
2	GET	`~/audit-events/{id}`	auditLog.view	—	AuditEvent + related events	`events`	R
3	POST	`~/audit-events/{id}/reveal`	Owner	field, reason	unmasked value (the reveal is itself logged)	`reveal`	R
4	GET	`~/audit-events/verify`	auditLog.view	from?, to?	chain ok / first broken id	`chain check`	R

Rules the server must enforce

- Append-only: no update or delete endpoint exists. Each entry stores `previousHash` and `hash` (SHA-256 over the canonical entry + previous hash).
- Each entry: who (name, role), what (action, entity, before/after changes), when, scope (branch), reason, source (web | import | system | kiosk), masked IP.
- Contact details, IDs, dates of birth and IPs are masked when read or exported.

5.11 Reports, exports and dashboards (6)

Replaces `ReportRepository`. Today reports are built in the app from loaded data; a backend should build them.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	`~/reports/{kind}`	reports.view + module view	from, to, branchId?, department?, columns[]	ReportTable (headers, rows, totals, note)	`client today`	N
2	POST	`~/exports`	see export rule	source (report kind, employees, attendance, leaveRequests, leaveBalances, holidays, audit), filters, columns, format (csv | xlsx)	ExportRecord (status, download url)	`add`	R
3	GET	`~/exports`	reports.view	cursor	ExportRecord[] (own scope)	`exports`	R
4	GET	`~/exports/{id}/download`	requester or same scope	—	file (logged)	`download`	R
5	GET	`~/dashboard/overview`	reports.view	branchId?, period (last14 | thisMonth | last30)	DashboardMetrics + attendance trend + leave by type	`client today`	N
6	GET	`~/dashboard/branch/{branchId}`	reports.view (scoped)	date?	today by department, coverage, pending leave	`client today`	N

Rules the server must enforce

- Report kinds: roster, changes, daily, monthly, missing, balances, requests, holidays, audit (audit: Owner and HR Admin only).
- Export needs: module view + export, reports.export for reports, the Data & export right for the area (people, attendance, leave), and for restricted columns (mobile, personal email) the restricted right plus organization scope.
- Rows limited to the requester's scope. Every export and every download is audited. Files expire after `linkDays` / `exportFileDays`.
- Use the organization's data settings: date format (dmy | iso), durations (hoursMinutes | decimal), CSV separator.

5.12 Platform statistics (super admin) (3)

Statistics-only operator view (D-75, boards S97–S98). Replaces `PlatformRepository`. Not under `/orgs/{orgId}`: the caller has no organization.

#	Method	Path	Permission	Input	Output	Replaces	
1	GET	`/platform/overview`	Platform admin	—	Platform totals, daily active (30 days), sign-ups per month (12), feature use, counts by industry and state (no leave-request tile)	`snapshot`	R
2	GET	`/platform/organizations`	Platform admin	search?, status (active | settingUp | quiet)?, sort, page	Page<OrgStats> (no branches)	`snapshot`	R
3	GET	`/platform/organizations/{orgId}`	Platform admin	—	OrgStats with BranchStats[] and dailyActive[30]	`snapshot`	R

Rules the server must enforce

- The platform admin is an account flag (`isPlatformAdmin`) with two-step always on and **no** organization membership; every `/orgs/{orgId}/*` endpoint returns 403 for it, and `/platform/*` returns 403 for everyone else.
- **Aggregates only:** responses may contain counts, percentages and the organization's own business details (name, ID, industry, city, state, size range, created date). They must never contain owner or employee names, emails, phones, IDs, photos or individual records. Build the responses from aggregate queries or a reporting view, not by filtering full records in code.
- Status: settingUp when an organization has no employed people or no fully set-up active branch; quiet when no punch in 7 days; otherwise active.
- Every platform request is written to a platform audit log (who, when, which endpoint).

5.13 Files (1)

Uploads used by organization logo, owner photo and employee photos.

#	Method	Path	Permission	Input	Output	Replaces	
1	POST	`~/uploads`	Signed in	multipart file, purpose (logo | ownerPhoto | employeePhoto | employeeImport | holidayImport)	uploadId, url	`file_picker uploads`	R

Rules the server must enforce

- Images: PNG or JPG only, ≤ 2 MB, checked by the file's first bytes (renamed files refused); SVG refused. Imports: .xlsx or .csv.

- Scan uploads for malware; serve images from signed URLs.

6. Data models

Types: `string, int, number` (decimal), `bool, date` (YYYY-MM-DD), `datetime` (ISO 8601 UTC), `enum`, `[]` array, `?` nullable. Shared enums: **SystemRole** = owner | hrAdmin | branchManager | attendanceManager | leaveApprover | employee. **DataScope** = organization | assignedBranches | ownTeam | ownRecords. **NotifyEvent** = leaveSubmitted | leaveDecided | leaveWithdrawn | missingPunch | correctionDecided | repeatedLateness | faceFlagged | faceEnrolSubmitted | faceDecided | invitation | profileChanged | inviteBounced | holidaySoon | holidayAdded. **ReportKind** = roster | changes | daily | monthly | missing | balances | requests | holidays | audit.

UserAccount

Field	Type	Notes
`id`	string	Account id
`fullName`	string	
`email`	string	
`employeeId`	string?	Employee code when linked, e.g. EMP-0229

OrganizationMembership

Field	Type	Notes
`organizationId`	string	
`organizationName`	string	
`role`	enum SystemRole	
`branchIds`	string[]	Empty = all
`scopeLabel`	string	e.g. "Surat, Vadodara"
`summary`	string	e.g. "3 branches · 248 employees"
`status`	enum	active | deactivated
`lastUsed`	bool	

AppSession

Field	Type	Notes
`userId`	string	
`displayName`	string	
`email`	string	
`organizationId / organizationName`	string	
`role`	enum SystemRole	Base role (sets menu and home)
`roleId / roleTitle`	string?	Custom role
`grants`	string[]	Permission keys "module.action"
`scope`	enum DataScope	
`scopeLabel`	string	
`branchIds`	string[]	
`employeeId`	string?	
`canSwitchOrganization`	bool	

Invitation

Field	Type	Notes
`token`	string	
`inviterName`	string	
`organizationName`	string	
`role`	enum SystemRole	
`branchName`	string	
`fullName`	string	

Field	Type	Notes
employeeId	string	
email	string	
startDate	date	
expiresAt	datetime	

Organization

Field	Type	Notes
id	string	
name	string	Display name
employeeRange	string	e.g. "201–500"
logoUrl	string?	Upload
legalName	string	
industry	string	
registrationNumber	string	Masked when read
contactEmail / contactPhone	string	
address, city, state, pin, country	string	
timeZone	string	IANA, default Asia/Kolkata
createdAt / updatedAt	datetime	
updatedBy	string	
version	int	Optimistic locking

OwnerProfile

Field	Type	Notes
fullName, email, phone, designation	string	
photoUrl	string?	

SetupBookmark

Field	Type	Notes
branchId	string	
step	string	basics | attendance | holidays | leave | roles | reminders | review

Branch

Field	Type	Notes
id	string	
name	string	Unique
code	string	Unique, e.g. SUR-01
address, city, state	string	
pin	string	6 digits
phone, email	string	
timeZone	string?	Null = inherit
hours	DayHours[7]	Mon–Sun: {open, opensAt, closesAt} minutes after midnight
status	enum	active | inactive
employeeCount	int	Computed

Field	Type	Notes
attendance	AttendanceRules?	{method: app|kiosk|both, graceMinutes 10, earlyLeaveMinutes 30, halfDayLateMinutes 120, shiftMode: branchHours|several, selfieCheck: false}
holidays	HolidaySetup?	{source: gujarat|national|none, items: HolidayItem[]}
leave	LeaveSetup?	{types: [{key, name, daysPerYear, accrual, enabled, tracksBalance}], approval: manager|managerThenHr|owner}
roles	RolesSetup?	{manager: owner|invite, managerEmail, managerCanApproveLeave, managerCanCorrectAttendance, managerCanSeeReports, managerCanEditPeople, employeesCanRequestLeave, employeesCanSeeTeamOff}
reminders	ReminderSettings?	{punchInReminder, missingPunchSummary, weeklyLeaveDigest}
reviewed	bool	Setup review done
openedOn	date?	
version	int	

HolidayItem (branch)

Field	Type	Notes
name	string	
date	date	
observed	bool	
custom	bool	
everyYear	bool	
category	enum	festival | national | regional | company
optional	bool	
orgWide	bool	
description	string	

Employee

Field	Type	Notes
id	string	
code	string	Employee ID, unique
firstName, lastName, displayName	string	
workEmail	string	Unique
mobile	string	Restricted
personalEmail	string	Restricted
dateOfBirth	date?	Masked
emergencyContact	string	
designation	string	
department	string	From departments list
type	enum	fullTime | partTime | contract | trainee
joiningDate	date	
probationMonths	int	
status	enum	active | probation | noticePeriod | inactive | terminated
statusReason	string	
lastWorkingDay	date?	
branchId	string	
workLocation	string	

Field	Type	Notes
managerId	string?	Employee id
shift	string	Branch shift name
weeklyOff	int?	1 = Mon … 7 = Sun; personal override
accountStatus	enum	signedIn | invited | inviteExpired | noAccount
role	enum SystemRole	
invitedAt / lastSignInAt	datetime?	
customValues	object	key → value
hasAttendance	bool	
activity	EmployeeActivity[]	{text, kind, by, at}
version	int	

EmployeeFieldSettings

Field	Type	Notes
rules	object	field → required | optional | hidden; only changed fields. Fields: displayName, workEmail, mobile, personalEmail, dateOfBirth, emergencyContact, designation, department, probation, workLocation, weeklyOff (others locked)
updatedAt / updatedBy	datetime / string	
version	int	

CustomField

Field	Type	Notes
id, key	string	key unique
label, description	string	
type	enum	shortText | longText | number | date | singleSelect | multiSelect | yesNo | url
options	string[]	For selects
required, sensitive, archived	bool	
departments, branchIds	string[]	Empty = all
access	object	audience (hrAdmin | branchManager | attendanceManager | self) → {view, edit}
createdAt	datetime?	

ImportRecord

Field	Type	Notes
id	string	
fileName	string	
finishedAt	datetime	
by	string	
created, updated, skipped, failed	int	
branchIds	string[]	
invitationsSent	int	
notImported	NotImportedRow[]	{rowNumber, name, code, email, outcome: created|updated|skipped|failed, reason}
createdEmployeeIds	string[]	

Punch

Field	Type	Notes
id	string	
employeeId	string	
at	datetime	Server UTC
kind	enum	punchIn | punchOut | breakStart | breakEnd
source	enum	app | kiosk | web | correction
correctionId	string?	
replaces	string?	Punch id

Correction

Field	Type	Notes
id	string	
employeeId	string	
workDate	date	
changes	array	{kind, at, replaces?}
reason	string	
by / byEmployeeId	string	
createdAt	datetime	
status	enum	pending | approved | rejected
approverNote	string	
decidedBy / decidedAt	string? / datetime?	
requestId	string?	CorrectionRequest answered

CorrectionRequest

Field	Type	Notes
id	string	
employeeId	string	
workDate	date	
kind	enum PunchKind	
proposedAt	datetime	
message	string	
createdAt	datetime	
status	enum	open | resolved | rejected
rejectReason	string	

DayRecord (calculated)

Field	Type	Notes
employeeId	string	
date	date	
status	enum	present | late | leftEarly | halfDay | shortShift | missingPunch | absent | onLeave | holiday | weeklyOff | working | notYet | upcoming | none
sessions	array	{in, out, breaks[]}
shift	object?	{start, end}
workedMinutes, breakMinutes, lateMinutes, earlyMinutes	int	
scheduled	bool	
holidayName	string?	

Field	Type	Notes
corrected	bool	
pendingCorrection	Correction?	
correctedBy / correctionReason	string?	

FaceCheck (on Punch.face)

Field	Type	Notes
result	enum	verified | notMatched | noFace | notEnrolled
score	number?	Similarity 0–1
photoUrl	string?	Signed URL, reviewers and the employee only
review	FaceReview?	

FaceReview

Field	Type	Notes
punchId	string	
accepted	bool	
by	string	
at	datetime	
note	string	Required when not accepted

FaceProfile

Field	Type	Notes
employeeId	string	
status	enum	pending | approved | rejected
photoUrls	string[3]	Straight, left, right
consentAt	datetime	
submittedAt	datetime	
decidedBy / decidedAt	string? / datetime?	
rejectReason	string	

FaceExemption

Field	Type	Notes
employeeId	string	
reason	string	
by	string	
at	datetime	

LeaveType

Field	Type	Notes
key	string	e.g. casual
name	string	
color	int	ARGB
paid	bool	
method	enum	upfront | monthly | none
allowance	number	Days per year
startsAfterMonths	int	
carryForwardMax	number	

Field	Type	Notes
minDays / maxDays	number	
allowHalfDays, countWeeklyOffs, countHolidays	bool	
noticeDays, backdateDays	int	backdate ≤ 30
eligibleTypes	enum[]	Employment types
branchIds, departments	string[]	Empty = all
active	bool	
optionalHolidaysOnly	bool	

LeavePolicy

Field	Type	Notes
routes	ApprovalRoute[]	{id, branchId?, typeKey?, second: none|hrIfDays|branchManagerIfDays|hrAlways, minDays}
backdatedSickDays	int	
longestRequestDays	int	
blackouts	Blackout[]	{name, from, to, departments[], branchIds[], blockedTypes[]}
allowNegative	bool	
negativeLimit	number	
cancelApprovedNotStarted	bool	
optionalHolidaysPerYear	int	Default 1
updatedAt / updatedBy		
version	int	

LeaveRequest

Field	Type	Notes
id	string	e.g. L-2514
employeeId	string	
typeKey	string	
from / to	date	
half	enum	full | first | second
days	number	Calculated
reason	string	
createdAt	datetime	
status	enum	pending | changesRequested | approved | rejected | withdrawn | cancelled
steps	ApprovalStep[]	{kind: manager|branchManager|hr, approverId?, note, decidedBy?, decidedAt?, approved?}
history	array	{at, text, by}
decisionNote	string	
amendedFrom	object?	{from, to}

LeaveAdjustment

Field	Type	Notes
id	string	
employeeId	string	
typeKey	string	
days	number	+ or −

Field	Type	Notes
effective	date	
reason	string	Required
by	string	
carry	bool	Carried from last year

LeaveBalance (calculated)

Field	Type	Notes
typeKey	string	
granted, carried, adjusted, used, pending, available	number	
offered	bool	Type applies to this person

CompanyHoliday

Field	Type	Notes
id	string	
name	string	
date / endDate	date	Range ≤ 14 days
category	enum	festival | national | regional | company
branchIds	string[]	Empty = all
optional	bool	
description	string	
createdBy	string	

ScheduleAssignment

Field	Type	Notes
id	string	
level	enum	employee | group | branch | organization
name	string	
effectiveFrom	date	Not in the past
branchId	string?	
employeeIds	string[]	
shift	string?	
weeklyOff	int?	

RoleDefinition

Field	Type	Notes
id	string	
name, description	string	
kind	enum	system | custom
base	enum SystemRole	"Works like"
scope	enum DataScope	
grants	string[]	"module.action" keys
active	bool	
updatedAt / updatedBy		
version	int	

RoleAssignment

Field	Type	Notes
key	string	Employee code or member key
roleId	string	
branchIds	string[]	
teamSize	int	Calculated
changedAt / changedBy		

SecuritySettings

Field	Type	Notes
twoStep	enum	optional | admins | everyone
minPasswordLength	int	8–64, default 10
lockAfterAttempts	int	
lockMinutes	int	Default 15
adminIdleMinutes	int	Default 30
employeeIdleDays	int	Default 30
inviteDays	int	Default 7
resetMinutes	int	Default 30

DataSettings

Field	Type	Notes
exportRights	object	role → [people | attendance | leave | restricted]
dateFormat	enum	dmy | iso
durations	enum	hoursMinutes | decimal
csvSeparator	string	
linkDays	int	Download link life
punchYears	int	Retention
leaverYears	int	Default 3
exportFileDays	int	Default 30

BrandingSettings

Field	Type	Notes
color	int	
language	string	
longDates, clock24, indianNumbers, weekStartsMonday	bool	

AppNotification

Field	Type	Notes
id	string	
recipient	string	Member key
category	enum	approvals | attendance | people | system
tone	enum	ok | fix | no | info | user
title, short, body	string	
facts	array	[label, value] pairs
at	datetime	
route	string?	App page to open
cta	string	Button label

Field	Type	Notes
read	bool	
event	enum NotifyEvent?	

NotificationPrefs

Field	Type	Notes
overrides	array	{event, to: employee|manager|hr, channel: inApp|email, on}
digest	bool	
digestAt	int	Hour
updatedAt	datetime?	

AuditEvent

Field	Type	Notes
id	string	
at	datetime	
actorName, actorRole	string	
action	string	e.g. employee.updated
entityType, entityId, entityLabel, entityRoute	string	
branchId	string?	
scopeLabel	string	
summary, reason	string	
source	enum	web | import | system | kiosk
ip	string	Masked
changes	array	{field, before, after}
affected, relatedIds	string[]	
previousHash, hash	string	Chain

OrgStats (platform)

Field	Type	Notes
id, name, industry, city, state, sizeRange	string	Organization business details only
createdAt	datetime	
employees, inactiveEmployees	int	
accountsSignedIn, accountsInvited	int	
activeToday	int	People with a punch today
dailyActive	int[30]	Oldest first
punches30, leaveRequests30, leavePending	int	
customFields, imports, faceEnrolled, selfieBranches	int	Feature use
lastActivity	datetime?	
status	enum	active | settingUp | quiet
branches	BranchStats[]	{id, name, code, city, active, employees, setupPercent, activeToday, selfieCheck}

ExportRecord

Field	Type	Notes
id	string	

Field	Type	Notes
`kind`	enum ReportKind	
`filters`	string	Human summary
`requestedBy / requestedById`	string	
`at`	datetime	
`format`	string	csv | xlsx
`failed`	bool	
`downloadUrl`	string?	Expires
`branchId`	string?	

7. Background jobs and server-side events

Job / event	When	What it does
Audit writer	Every mutation (same transaction)	Appends a sealed AuditEvent: actor, action, entity, before/after, reason, scope, source, masked IP, previousHash → hash.
Notification fan-out	After leave, attendance, invitation, profile and holiday events	Creates in-app notifications and queues emails per the preference matrix; skips the actor.
Email sender	Queue worker	Verification codes, invitations, password resets, notifications, digest. Records bounces (shown in Invitations as "Not delivered").
Leave accrual	1st of each month	Monthly accrual after the waiting months; or compute on read from rules (preferred: no stored balance).
Year-end carry-forward	1 January (leave year Jan–Dec)	Carries up to `carryForwardMax`, lapses the rest; writes adjustments with `carry = true`.
Missing-punch detection	After each shift end (branch time zone)	Flags open sessions; sends reminders if enabled; marks > 3 days as overdue.
Daily digest	Organization `digestAt` hour	Summary email for managers/HR if digest is on.
Invitation expiry	Hourly	Sets `accountStatus = inviteExpired` after `inviteDays`.
Export files	On request; cleanup daily	Generates CSV/XLSX in the background; deletes files after `exportFileDays`; links expire after `linkDays`.
Retention	Daily	Removes punch data older than `punchYears` and leavers older than `leaverYears` (keeping audit entries).
Holiday reminders	Daily	"Holiday next week" notifications per branch.
Selfie clean-up	Daily	Deletes punch selfies older than 90 days; deletes face data of people who left.

8. Build order and integration checklist

Phase	Build	App screens that start working
1	Auth & session, uploads, organization, branches, settings (read)	Sign-in, two-step, register, onboarding, organization and branch setup
2	Employees, custom fields, invitations, import	Directory, profiles, add/edit, import wizard, invitations
3	Attendance (punch, corrections, requests) + day calculation	Employee home, my attendance, attendance pages
4	Holidays & schedules, leave (types, policy, requests, decisions, balances)	Leave and holiday pages, approvals
5	Roles, audit log, notifications, settings (write)	Roles & permissions, audit, notifications, settings
6	Reports, exports, dashboards, background jobs	Overview, branch dashboard, reports, export history

- For each module, the app developer replaces `Demo…Repository` with an HTTP version of the same interface and runs the existing 663 tests plus `integration_test/` against a seeded test server.
- Seed the test server with the demo organization (Meridian Textiles, 4 branches, 248 employees, six demo accounts) so the QA rule book (`docs/qa/Staffline_Rule_Book_QA.pdf`) applies unchanged.
- Contract tests: every endpoint has a test for its permission (allowed and refused role), its scope (out-of-scope record returns 404) and each validation message.
- Build the production app with `--dart-define=SHOW_DEV_TOOLS=false`.

9. Decisions the backend team must make

Topic	Notes
Stack and database	Any stack; a relational database (PostgreSQL) fits: organization-scoped tables, row-level security per organization recommended.
Email provider	Transactional email with bounce webhooks (feeds "Not delivered" invitations).
File storage	Object storage with signed URLs; malware scan on upload.

Topic	Notes
Two-step	TOTP (RFC 6238) with 10 backup codes; trusted-device cookie 30 days.
Time	Server clock in UTC; IANA time-zone database kept up to date.
Audit hash	SHA-256 chain per organization (the demo uses FNV-64).
Platform administration	Statistics endpoints are listed (5.12). Support sessions, billing and suspend from boards S97–S98 are not approved.
Kiosk punching, location checks	Not designed yet; `source = kiosk` is reserved.
Face matching service	Choose a face recognition engine with liveness detection (self-hosted model or a cloud API that keeps data in India), and set the match threshold from a pilot. Biometric data needs a DPDP-compliant consent record and a data-protection impact assessment.